

# Assignment 8

## Overview

**Advanced Embedded Software  
Development**  
with **Dan Walkes**



University of Colorado **Boulder**

**Learning objectives:**

Understand Assignment 8 requirements

# Assignment 8

# Assignment 8 Overview

- Standalone driver test/implementation

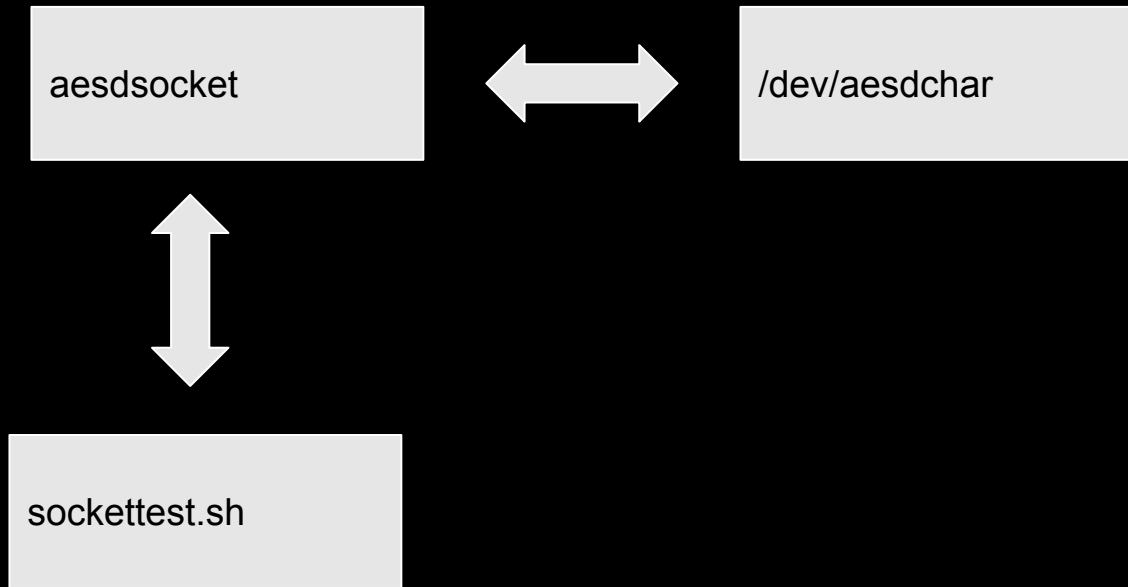
/dev/aesdchar



drivertest.sh

# Assignment 8 Overview

- Testing with  
sockettest.sh  
(replacement for  
/var/tmp/aesdso  
cketdata)



# Dynamic Allocation and Circular Buffer

- Our unit test for assignment 7 used const strings with no dynamic allocation
- The implementation for assignment 8 will need to allocate memory dynamically.
- When will you need to free memory from the circular buffer?
  - When adding to the circular buffer when the circular buffer was already full
- How can you communicate this to the caller?
  - Could return a pointer with buffer that was removed

# Dynamic Allocation and Circular Buffer

## Implementation Suggestion

```
/**
 * Adds entry @param add_entry to @param buffer in the location specified in buffer->in_offs.
 * If the buffer was already full, overwrites the oldest entry and advances buffer->out_offs to the
 * new start location.
 * Any necessary locking must be handled by the caller
 * @return NULL or, if an existing entry at out_offs was replaced,
 *         the value of buffptr for the entry which was replaced (for use with dynamic memory allocation/free)
 */
const char *aesd_circular_buffer_add_entry(struct aesd_circular_buffer *buffer, const struct aesd_buffer_entry *add_entry)
```

```
/**
 * Write the packet specified in @param writestr to the buffer specified
 * by @param buffer
 */
static void write_circular_buffer_packet(struct aesd_circular_buffer *buffer,
                                         const char *writestr)
{
    struct aesd_buffer_entry entry;
    const char * rtnptr;
    const char * start_out_ptr = buffer->entry[buffer->out_offs].buffptr;
    bool buffer_was_full = buffer->full;
    entry.buffptr = writestr;
    entry.size = strlen(writestr);
    rtnptr = aesd_circular_buffer_add_entry(buffer, &entry);
    if( buffer_was_full ) {
        TEST_ASSERT_EQUAL_PTR_MESSAGE(start_out_ptr, rtnptr, "Expect to return the value of output pointer when the buffer is full");
    } else {
        TEST_ASSERT_NULL_MESSAGE(rtnptr, "Expect to return NULL when the buffer was not full before add");
    }
}
```

Modified test case to test this logic

# aesd-circular-buffer.c/.h

```
#ifdef __KERNEL__
#include <linux/string.h>
#else
#include <string.h>
#endif

#include "aesd-circular-buffer.h"
```

```
#ifndef AESD_CIRCULAR_BUFFER_H
#define AESD_CIRCULAR_BUFFER_H

#ifdef __KERNEL__
#include <linux/types.h>
#else
#include <stddef.h> // size_t
#include <stdint.h> // uintx_t
#include <stdbool.h>
#endif
```

- What do these lines do?
  - Use appropriate header files for types so compile works in either unit test or kernel context
  - Use the appropriate section when adding any header files for your kernel driver implementation
    - Will mean you can still run unit tests



# aesdchar.h

```
#ifndef AESD_CHAR_DRIVER_AESDCHAR_H_
#define AESD_CHAR_DRIVER_AESDCHAR_H_

#define AESD_DEBUG 1 //Remove comment on this line to enable debug

#undef PDEBUG          /* undef it, just in case */
#ifdef AESD_DEBUG
#  ifdef __KERNEL__
    /* This one if debugging is on, and kernel space */
#    define PDEBUG(fmt, args...) printk( KERN_DEBUG "aesdchar: " fmt, ## args)
#  else
    /* This one for user space */
#    define PDEBUG(fmt, args...) fprintf(stderr, fmt, ## args)
#  endif
#else
#    define PDEBUG(fmt, args...) /* not debugging: nothing */
#endif
```

- Debug prints are useful, please use them

# aesdchar.h

```
struct aesd_dev
{
    /**
     * TODO: Add content needed to complete assignment
     */
    struct cdev cdev;    /* Char device structure */
};
```

- What might you include in `aesd_dev` from your assignment 7 code?
  - circular buffer structure/code
  - working `aesd_buffer_entry` value (for writes before `\n`)
- What else might you need to add?
  - locking primitive

# init\_module

```
int aesd_init_module(void)
{
    dev_t dev = 0;
    int result;
    result = alloc_chrdev_region(&dev, aesd_minor, 1,
        "aesdchar");
    aesd_major = MAJOR(dev);
    if (result < 0) {
        printk(KERN_WARNING "Can't get major %d\n", aesd_major);
        return result;
    }
    memset(&aesd_device, 0, sizeof(struct aesd_dev));

    /**
     * TODO: initialize the AESD specific portion of the device
     */

    result = aesd_setup_cdev(&aesd_device);

    if( result ) {
        unregister_chrdev_region(dev, 1);
    }
    return result;
}

module_init(aesd_init_module);
```

```
struct file_operations aesd_fops = {
    .owner =    THIS_MODULE,
    .read =    aesd_read,
    .write =    aesd_write,
    .open =    aesd_open,
    .release =  aesd_release,
};
```

```
static int aesd_setup_cdev(struct aesd_dev *dev)
{
    int err, devno = MKDEV(aesd_major, aesd_minor);

    cdev_init(&dev->cdev, &aesd_fops);
    dev->cdev.owner = THIS_MODULE;
    dev->cdev.ops = &aesd_fops;
    err = cdev_add (&dev->cdev, devno, 1);
    if (err) {
        printk(KERN_ERR "Error %d adding aesd cdev", err);
    }
    return err;
}
```

# cleanup\_module

```
void aesd_cleanup_module(void)
{
    dev_t devno = MKDEV(aesd_major, aesd_minor);

    cdev_del(&aesd_device.cdev);

    /**
     * TODO: cleanup AESD specific portions here as necessary
     */

    unregister_chrdev_region(devno, 1);
}
```

# open

```
int aese_open(struct inode *inode, struct file *filp)
{
    PDEBUG("open");
    /**
     * TODO: handle open
     */
    return 0;
}
```

- filp - file pointer
  - Set filp->private\_data with our aese\_dev device struct
- How do we get a pointer to our aese\_dev struct?
  - Use inode->i\_cdev with container\_of to locate within

aese\_dev

```
struct aese_dev
{
    /**
     * TODO: Add content needed to complete assignment
     */
    struct cdev cdev;      /* Char device structure */
};
```

# file\_operations read

```
ssize_t aead_read(struct file *filp, char __user *buf, size_t count,  
                 loff_t *f_pos)  
{
```

- filp - file pointer
  - private\_data member can be used to get aead\_dev
- buff - buffer to fill
  - Can we access this buffer directly?
    - No - use copy\_to\_user
- count - max number of bytes to write to buff
  - You may want/need to write less than this

# file\_operations read

```
ssize_t aead_read(struct file *filp, char __user *buf, size_t count,  
                  loff_t *f_pos)  
{
```

- `f_pos` - pointer to the read offset
  - references a location in your virtual device
  - In our case, a specific byte of the circular buffer linear content.  
Which value?
    - `char_offset`
  - Start the read at this offset
  - Update the pointer to point to the next offset

# read return values

```
ssize_t read(struct file *filp, char __user *buf, size_t count,  
             loff_t *f_pos)  
{
```

- If return == count
  - requested number of bytes were transferred
- If  $0 < \text{return} < \text{count}$ 
  - only a portion has been returned (see partial read rule in next slide)
- 0 - end of file (no data read into buff)
- Negative - error (-ERESTARTSYS, -EINTR, -EFAULT)



# “partial read rule”

```
ssize_t aead_read(struct file *filp, char __user *buf, size_t count,  
                 loff_t *f_pos)  
{
```

- Each read can optionally return a portion of the total data available
  - For instance, only a single Assignment 8 write command and circular buffer entry
- As long as return values is set correctly and offset (\*offp) is updated correctly, the application can retry the read to read all data until all available data is read.
  - Happens automatically with fread (or cat)

# file\_operations write

```
ssize_t aed_write(struct file *filp, const char __user *buf, size_t count,  
                  loff_t *f_pos)  
{
```

- Typically f\_pos would point to the location where a write should start
  - In Assignment 8 we will ignore and always either:
    - append to the command being written when there's no newline received or
    - write to the command buffer when a newline is received

# write return values

```
ssize_t aead_write(struct file *filp, const char __user *buf, size_t count,  
                  loff_t *f_pos)  
{
```

- return value == count, requested number of bytes written successfully
- Positive but smaller than count, only part written, may retry
- 0 - nothing written, may retry
  - Why is my application hanging attempting to perform the first write?
- Negative - error occurred -ENOMEM, -EFAULT

# drivertest.sh

- Each echo will write one newline terminated command

```
device=/dev/aesdchar  
  
echo "write1" > ${device}  
echo "write2" > ${device}  
echo "write3" > ${device}  
echo "write4" > ${device}  
echo "write5" > ${device}  
echo "write6" > ${device}  
echo "write7" > ${device}  
echo "write8" > ${device}  
echo "write9" > ${device}  
echo "write10" > ${device}
```

# drivertest.sh

```
cat >${expected_file} << EOF
write1
write2
write3
write4
write5
write6
write7
write8
write9
write10
EOF
```

- Build an expected file containing expected content
- << EOF syntax is known as heredoc

# drivertest.sh

```
check_output()
{
    local read_file=$1
    local expected_file=$2
    diff ${read_file} ${expected_file}
    if [ $? -ne 0 ]; then
        echo "difference detected, expected:"
        cat ${expected_file}
        echo "but found"
        cat ${read_file}
        rc=-1
    fi
}
```

```
check_output ${read_file} ${expected_file}
```

- Compare differences between \${read\_file} returned by your driver and \${expected\_file} created with heredoc

# drivertest.sh

```
echo "write11" > ${device}
expected_file_2_to_11=$(mktemp)
cat >${expected_file_2_to_11} << EOF
write2
write3
write4
write5
write6
write7
write8
write9
write10
write11
EOF
```

- Writing an 11th command should rotate out the first command from the output list (based on circular buffer implementation)

# drivertest.sh

- Writing without newline (echo -n) should mean multiple writes are still considered a single command

```
echo -n "w" > ${device}
echo -n "r" > ${device}
echo -n "it" > ${device}
echo "e1" > ${device}
echo "write2" > ${device}
echo "write3" > ${device}
echo "write4" > ${device}
echo "write5" > ${device}
echo "write6" > ${device}
echo "write7" > ${device}
echo "write8" > ${device}
echo "write9" > ${device}
echo -n "wr" > ${device}
echo -n "it" > ${device}
echo "e10" > ${device}
```

```
cat >${expected_file} << EOF
write1
write2
write3
write4
write5
write6
write7
write8
write9
write10
EOF
```